

assignmentD

DSA Assignment D: 20260527模拟考

Updated 2026-05-31 18:04 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M27351:01最小生成树

补图的连通分量, <http://cs101.openjudge.cn/practice/27351>

思路：

补图连通块

这是解决本题最关键的技巧。

- **思想：**由于 0 权边是“隐式”存在的（即不在输入中的边），我们不能直接遍历 0 权边。代码通过维护 `not_visited` 集合，记录尚未分配到连通块的点。
- **技巧：**在 BFS 过程中，对于当前点 `u`，所有不在 `edges[u]` 中的点 `v` 都与 `u` 有 0 权边相连。
- **复杂度分析：**
 - 外层 `while queue` 保证每个点只作为 `u` 出现一次。

- 内层 `for v in not_visited` 虽然看起来次数很多，但由于一旦找到 `v` 满足条件，就立刻将其从 `not_visited` 中 `remove` 掉。
- **关键点：** `if v not in forbidden` 的判断次数总计大约是 $O(V + E)$ 。这是因为对于每个点，它只会跳过那些在 `edges[u]` 里的点，而所有 `edges[u]` 的长度之和是 $2E$ 。

为什么用 BFS 而不用 DFS：更新 `not_visited` 时，需要将每次遍历到的 `v` 在这层就删掉，而不能带入下一轮扩展（否则时间复杂度会不满足），而 BFS 很适合这种分层处理。而用 DFS 每一轮都需要 `copy not_visited` 集合，这一层的遍历的点集可能与下一层的点集重复，在时间和空间复杂度上都难以满足。

代码：

```
import sys
from collections import deque

def bfs(s, edges, not_visited):
    q = deque([s])
    while q:
        u = q.popleft()
        tmp = set()
        for v in not_visited:
            if v not in edges[u]:
                tmp.add(v)
        not_visited -= tmp
        q.extend(tmp)

def main():
    input = sys.stdin.readline
    n, m = map(int, input().split())
    edges = [set() for _ in range(n + 1)]
    for _ in range(m):
        a, b = map(int, input().split())
        edges[a].add(b)
        edges[b].add(a)
    not_visited = set(range(1, n + 1))
    ans = 0
    while not_visited:
        u = not_visited.pop()
        bfs(u, edges, not_visited)
        ans += 1
    print(max(ans - 1, 0))

if __name__ == '__main__':
    main()
```

代码运行截图（至少包含有"Accepted"）

#52656202提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
import sys
from collections import deque

def bfs(s, edges, not_visited):
    q = deque([s])
    while q:
        u = q.popleft()
        tmp = set()
        for v in not_visited:
            if v not in edges[u]:
                tmp.add(v)
        not_visited -= tmp
        q.extend(tmp)

def main():
```

基本信息

#: 52656202
题目: 27351
提交人: 25n2500011619(xq)
内存: 61392kB
时间: 3488ms
语言: Python3
提交时间: 2026-05-28 16:18:22

M30910:邮递员送快递

正向/反向图 Dijkstra, <http://cs101.openjudge.cn/practice/30910>

思路:

从 1 号点到每个点的最短路径可以用 dijkstra 跑一遍正图。

从 2~n 号回来: dijkstra 不支持多起点, 一个个跑时间复杂度无法接受。采取的变通方法是将原图反转, 同样从 1 号点出发跑最短路, 对应的路径反过来就是回来的路径。

代码:

```
import sys
from heapq import heappop, heappush

def dijkstra(n, edges):
    dis = [float('inf')] * (n + 1)
    dis[1] = 0
    q = [(0, 1)]
    while q:
        disu, u = heappop(q)
        if disu > dis[u]:
            continue
        for v, w in edges[u]:
            tmp = dis[u] + w
            if tmp < dis[v]:
                dis[v] = tmp
                heappush(q, (tmp, v))
    return sum(dis[1:])
```

```
def main():
    input = sys.stdin.readline
    n, m = map(int, input().split())
    edges1 = [[] for _ in range(n + 1)]
    edges2 = [[] for _ in range(n + 1)]
    for _ in range(m):
        u, v, w = map(int, input().split())
        edges1[u].append((v, w))
        edges2[v].append((u, w))

    ans = dijkstra(n, edges1) + dijkstra(n, edges2)

    print(ans)

if __name__ == '__main__':
    main()
```

代码运行截图（至少包含有"Accepted"）

#52649268提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
import sys
from heapq import heappop, heappush

def dijkstra(n, edges):
    dis = [float('inf')] * (n + 1)
    dis[1] = 0
    q = [(0, 1)]
    while q:
        disu, u = heappop(q)
        if disu > dis[u]:
            continue
        for v, w in edges[u]:
            tmp = dis[u] + w
            if tmp < dis[v]:
                dis[v] = tmp
                heappush(q, (tmp, v))
```

基本信息

#: 52649268
 题目: 30910
 提交人: 25n2500011619(xq)
 内存: 25656kB
 时间: 207ms
 语言: Python3
 提交时间: 2026-05-27 18:32:42

M30912:累加树

构建 BST + 右-根-左累加 + BFS 输出, <http://cs101.openjudge.cn/practice/30912>

思路:

解决该问题的思路可以分为三个主要步骤:

1. **重建二叉搜索树 (BST)**: 由于输入的是二叉搜索树的先序遍历序列, 可以按照该序列的顺序依次将节点插入到树中, 从而还原出原二叉树的结构。
2. **转换为累加树 (GST)**: 累加树的定义是每个节点的值等于原树中大于或等于该节点值的所有节点值之和。在二叉搜索树中, 右子树的所有值都大于当前节点, 左子树的所有值都小于当

前节点。因此，可以通过**反向中序遍历**（右 -> 中 -> 左）的方式，从大到小遍历所有节点，并使用一个全局累加变量更新每个节点的值。

3. **层次遍历输出**：使用队列对转换后的累加树进行广度优先搜索（BFS），即可得到层次遍历序列。

代码：

```
import sys
from collections import deque

class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

sys.setrecursionlimit(10**4)
def layer(n, tree, sorted_tree):

    def build(l1, r1, l2, r2):
        if l1 > r1:
            return None
        node = Node(l1)
        if l1 == r1:
            return node
        pos = sorted_tree.index(tree[l1])
        left = build(l1 + 1, l1 + pos - l2, l2, pos - 1)
        right = build(l1 + pos - l2 + 1, r1, pos + 1, r2)
        node.left = left
        node.right = right
        return node

    root = build(0, n - 1, 0, n - 1)
    ans = []
    q = deque([root])
    while q:
        size = len(q)
        for _ in range(size):
            u = q.popleft()
            ans.append(u.val)
            if u.left:
                q.append(u.left)
            if u.right:
                q.append(u.right)
```

```

rank = [0] * n
for i, x in enumerate(ans):
    rank[x] = i
return rank

def main():
    n = int(input())
    tree = list(map(int, input().split()))
    sorted_tree = sorted(zip(range(n), tree), key=lambda x: x[1])
    suf_sum = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        suf_sum[i] = sorted_tree[i][1] + suf_sum[i + 1]

    ### pre to layer
    rank = layer(n, tree, sorted(tree))

    ans = [0] * n

    for i in range(n):
        ans[rank[sorted_tree[i][0]]] = suf_sum[i]

    print(' '.join(map(str, ans)))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52649238提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

import sys
from collections import deque

class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

sys.setrecursionlimit(10**4)
def layer(n, tree, sorted_tree):

    def build(l1, r1, l2, r2):
        if l1 > r1:
            return None
        node = Node(l1)
        l1 += 1

```

基本信息

#: 52649238
 题目: 30912
 提交人: 25n2500011619(xq)
 内存: 3720kB
 时间: 22ms
 语言: Python3
 提交时间: 2026-05-27 18:30:33

M30899:火星大工程

关键路径, <http://cs101.openjudge.cn/practice/30899>

思路:

AOE模板题

代码:

```
from collections import defaultdict

def topo_sort(n, edges, in_deg):
    q = [i for i in range(1, n + 1) if in_deg[i] == 0]
    est = [0] * (n + 1)
    head = 0
    while head < len(q):
        u = q[head]
        head += 1
        for v, w in edges[u]:
            in_deg[v] -= 1
            if est[u] + w > est[v]:
                est[v] = est[u] + w
            if in_deg[v] == 0:
                q.append(v)
    return est, q

def get_critical_path(n, edges, in_deg):
    est, topo_order = topo_sort(n, edges, in_deg)

    max_length = max(est)
    lst = [max_length] * (n + 1)
    activity = defaultdict(list)
    for u in reversed(topo_order):
        for v, w in edges[u]:
            if lst[v] - w < lst[u]:
                lst[u] = lst[v] - w
            if est[u] + w == lst[v]:
                activity[u].append(v)

    return max_length, activity

def main():
    n, m = map(int, input().split())
    edges = [[] for _ in range(n + 1)]
    in_deg = [0] * (n + 1)
    for _ in range(m):
```

```

    u, v, w = map(int, input().split())
    edges[u].append((v, w))
    in_deg[v] += 1

in_deg_copy = in_deg[:]

max_length, activity = get_critical_path(n, edges, in_deg_copy)

print(max_length)

for u in sorted(activity.keys()):
    for v in sorted(activity[u]):
        print(u, v)

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52673933提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

from collections import defaultdict

def topo_sort(n, edges, in_deg):
    q = [i for i in range(1, n + 1) if in_deg[i] == 0]
    est = [0] * (n + 1)
    head = 0
    while head < len(q):
        u = q[head]
        head += 1
        for v, w in edges[u]:
            in_deg[v] -= 1
            if est[u] + w > est[v]:
                est[v] = est[u] + w
            if in_deg[v] == 0:
                q.append(v)
    return est, q

def get_critical_path(n, edges, in_deg):

```

基本信息

#: 52673933
 题目: 30899
 提交人: 25n2500011619(xq)
 内存: 4264kB
 时间: 29ms
 语言: Python3
 提交时间: 2026-05-30 12:09:46

T30868:upstairs

同余最短路, <http://cs101.openjudge.cn/practice/30868>

思路:

这是典型的“同余最短路”模型。核心想法：只要知道“每个余数能达到的最小楼层”，就能判断任意 h 是否可达。

思路概述

- 设电梯每次只能上 a, b, c 层（允许为 0）。
- 取 $m = \min a, b, c$ 中的**最小正数**（如果存在）。
- 把“楼层”按 $\text{mod } m$ 取模，构造一个有 m 个点的图：点 r 表示当前楼层 $\equiv r(\text{mod } m)$ 。
- 从点 0 出发，每次上 a 层相当于从 r 走到 $(r + a) \text{mod } m$ ，边权为对应的步长。
- 用 Dijkstra 求出到每个余数的最小可达楼层 $\text{dist}[r]$ 。
- 对查询 h ，若 $\text{dist}[h \% m] \leq h$ ，则存在非负整数解使得达到 h （输出 Yes），否则 No。

为什么这样正确

- $\text{dist}[r]$ 是达到“余数为 r 的楼层”所需的最小楼层值。
 - 之后再加若干次“最小步长 m ”，不会改变余数，只会增高楼层。
 - 所以只要 h 不小于 $\text{dist}[h \% m]$ ，就能从 dist 再加若干个 m 达到 h 。
-

边界情况

1. **全部为 0：**
 - 只有 0 层可达。
 - 所有查询：若 $h=0$ 输出 Yes，否则 No。
 2. **只有一个或两个为 0：**
 - 仍按“最小正数”当作 m ，把 0 当作无效边（不添加）。
 3. **$m=1$ ：**
 - 所有楼层都可达。
-

复杂度

- 图有 m 个点，每点最多 3 条边。
 - Dijkstra： $O(m \log m)$ ， $m \leq 10^6$ 可接受。
 - 每个查询 $O(1)$ 。
-

实现步骤

1. 读入 a, b, c 和 q 。
2. 过滤出正数集合 $\text{steps} = [x \text{ for } x \text{ in } (a, b, c) \text{ if } x > 0]$ 。
3. 若 steps 为空：直接判定只有 $h=0$ 可达。
4. 令 $m = \min(\text{steps})$ ，建立 dist 数组大小为 m ，初始化为 INF。

5. Dijkstra:

- 起点 0, $\text{dist}[0]=0$ 。
- 从余数 r 走到 $(r+s)\%m$, 代价 $+s$ (s 为 a,b,c 且 >0)。

6. 对每个查询 h :

- 若 $\text{dist}[h \% m] \leq h$ 输出 Yes, 否则 No。

代码

```
import heapq
import sys

def dijkstra_min_remainder(steps):
    m = min(steps)
    dist = [10**30] * m
    dist[0] = 0
    pq = [(0, 0)]
    while pq:
        d, r = heapq.heappop(pq)
        if d != dist[r]:
            continue
        for s in steps:
            nr = (r + s) % m
            nd = d + s
            if nd < dist[nr]:
                dist[nr] = nd
                heapq.heappush(pq, (nd, nr))
    return m, dist

def main():
    data = sys.stdin.read().strip().split()
    if not data:
        return
    it = iter(data)
    a = int(next(it))
    b = int(next(it))
    c = int(next(it))
    q = int(next(it))

    steps = [x for x in (a, b, c) if x > 0]

    if not steps:
        out = []
        for _ in range(q):
```

```

        h = int(next(it))
        out.append("Yes" if h == 0 else "No")
    sys.stdout.write("\n".join(out))
    return

m, dist = dijkstra_min_remainder(steps)

out = []
for _ in range(q):
    h = int(next(it))
    out.append("Yes" if dist[h % m] <= h else "No")
sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

(至少包含有"Accepted")

#52661294提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

import heapq
import sys

def dijkstra_min_remainder(steps):
    m = min(steps)
    dist = [10**30] * m
    dist[0] = 0
    pq = [(0, 0)]
    while pq:
        d, r = heapq.heappop(pq)
        if d != dist[r]:
            continue
        for s in steps:

```

基本信息

#: 52661294
 题目: 30868
 提交人: 25n2500011619(xq)
 内存: 24868kB
 时间: 313ms
 语言: Python3
 提交时间: 2026-05-28 22:03:33

T30921:猫猫搭积木

并查集, <http://cs101.openjudge.cn/practice/30921>

思路:

用并查集维护连通块 (积木堆), `tot` 记录当前堆数。每次合并不同堆时 `tot -= 1`, 若合并后的大小 $k \geq s$ 则崩塌成 k 个单堆, 等价于 `tot += k - 1`。

可以用按秩合并 + 路径压缩

代码

```

import sys

def find(fa, x):
    y = x
    while fa[y] != y:
        y = fa[fa[y]]
    while x != y:
        z = fa[x]
        fa[x] = y
        x = z
    return y

def union(fa, size, children, x, y, s):

    if size[x] < size[y]:
        x, y = y, x
    children[x].append(y)
    fa[y] = x
    size[x] += size[y]

    res = 0
    if size[x] >= s:
        res = size[x] - 1
        q = [x]
        while q:
            u = q.pop()
            for v in children[u]:
                q.append(v)
            size[u] = 1
            fa[u] = u
            children[u] = []
    return res

def main():
    input = sys.stdin.readline
    n, q, s = map(int, input().split())
    output = []
    fa = list(range(n + 1))
    size = [1] * (n + 1)
    children = [[] for _ in range(n + 1)]
    tot = n
    for _ in range(q):
        a, b = map(int, input().split())
        a = find(fa, a)
        b = find(fa, b)
        if a != b:

```

```

        tot -= 1
        tot += union(fa, size, children, a, b, s)

    output.append(tot)

sys.stdout.write('\n'.join(map(str, output)))

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52674093提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

import sys

def find(fa, x):
    y = x
    while fa[y] != y:
        y = fa[fa[y]]
    while x != y:
        z = fa[x]
        fa[x] = y
        x = z
    return y

def union(fa, size, children, x, y, s):
    if size[x] < size[y]:
        x, y = y, x
    children[x].append(y)
    fa[x] = x

```

基本信息

#: 52674093
 题目: 30921
 提交人: 25n2500011619(xq)
 内存: 75924kB
 时间: 1382ms
 语言: Python3
 提交时间: 2026-05-30 12:34:46

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

本次模考AC3，第4题没猜到数据有问题，浪费了一些时间。T5 T6没时间做了，但是想到了T6按秩合并+暴力拆散的方法。从这几次的考试也积累了一些考场经验，不要在反复错的题目上消耗太多时间，后面标T的题目一定要看，未必没有想法。

本次模考题目总体还是比较难的，第一题用补图联通分量的想法，邮递员送快递这题从起点跑反图 = 从多个终点返回的想法都比较巧妙。回去学习了一下同余最短路算法，其算法核心就是最短路(SPFA/Dijkstra)，用到的数学知识也仅限于同余，严格来说不算超纲但是第一次做就是想不出来（苦笑）。

02442: Sequence

Heap, Greedy, <http://cs101.openjudge.cn/practice/02442/>

开始想的另一种合并方法，对于每一个组合 $s = A[i] + B[j]$ ，只有当 $i \times j \leq n$ 时才有可能是前 n 个最小和，那么候选的组合对数缩小至 $O(n \log n)$ 个，可以用堆或者 `sorted(res)[:n]` 的方法取出前 n 个最小和。当然这种方法时间复杂度更高，为 $O(m \cdot n \log^2 n)$ 。

更优的思路：

1. **两两合并**：由于总组合数 n^m 非常巨大（ 2000^{1000} ），直接生成是不现实的。我们可以采用两两合并的策略。先找前两个序列合并后的最小 n 个和，将结果作为一个新的“序列”，再与第三个序列合并，以此类推。
2. **双序列合并算法**：假设有两个已排序的序列 A 和 B，大小均为 n 。我们想求它们组合出的 n^2 个和中的前 n 小。
 - 首先对 A 和 B 进行排序。
 - 使用最小堆（Min-Heap）来维护候选和。
 - 初始时，将 $(A[i] + B[0], 0)$ 放入堆中（其中 $i \in [0, n - 1]$ ）。
 - 每次从堆顶取出最小的和 $s = A[i] + B[j]$ ，这就是当前的第 k 小和。
 - 取出后，如果 $j + 1 < n$ ，则将下一个候选和 $A[i] + B[j + 1]$ 放入堆中。
 - 重复上述过程 n 次，即可得到 A 和 B 合并后的前 n 个最小和。
3. **复杂度分析**：
 - 总共有 $m - 1$ 次合并操作。
 - 每次合并涉及 n 次堆操作，每次堆操作复杂度为 $O(\log n)$ 。
 - 总时间复杂度为 $O(m \cdot n \log n)$ 。

```
import sys
from heapq import heapreplace

def main():
    input = sys.stdin.readline
    outline = []
    for _ in range(int(input())):
        m, n = map(int, input().split())
        res = sorted(list(map(int, input().split())))
        q = []
        for _ in range(m - 1):
            row = sorted(list(map(int, input().split())))
            h = [(res[i] + row[0], 0) for i in range(n)]
            # heapify(h) unnecessary
            for i in range(n):
                curr_sum, idx = h[0]
                res[i] = curr_sum
                if idx + 1 < n:
                    nxt_sum = curr_sum + row[idx + 1] - row[idx]
                    heapreplace(h, (nxt_sum, idx + 1))
```

```

        outline.append(' '.join(map(str, res)))

    sys.stdout.write("\n".join(outline))

if __name__ == '__main__':
    main()

```

30909 最优爬山路径

Binary, BFS <http://cs101.openjudge.cn/practice/30909/>

二分 H_limit + BFS 判断

```

from collections import deque
import sys
def main():
    dr = [1, 0, -1, 0]
    dc = [0, 1, 0, -1]
    def bfs_check(h_limit):
        q = deque([(0, 0)])
        nonlocal bfs_id
        bfs_id += 1
        while q:
            ur, uc = q.popleft()
            if ur == n - 1 and uc == m - 1:
                return True
            hu = h[ur][uc]
            for t in range(4):
                vr, vc = ur + dr[t], uc + dc[t]
                if 0 <= vr < n and 0 <= vc < m:
                    idx = vr * m + vc
                    hv = h[vr][vc]
                    if vis[idx] != bfs_id and abs(hv - hu) <= h_limit:
                        vis[idx] = bfs_id
                        q.append((vr, vc))
            return False
    input = sys.stdin.readline
    n, m = map(int, input().split())
    h = [list(map(int, input().split())) for _ in range(n)]
    vis = [0] * (n * m)
    bfs_id = 0
    l = -1
    r = max(map(max, h))
    while r - l > 1:
        mid = (l + r) >> 1

```

```
        if bfs_check(mid): r = mid
        else: l = mid
    print(r)

if __name__ == '__main__':
    main()
```